Lab 2: Debugging with CubeIDE for Fibonacci Number Generation

Jianhua Liu

Spring 2026

Contents

2.1 Introduction . . . 2
- 2.1.1 The Fibonacci sequence . . . 2
- 2.1.2 An algorithm for generating part of the Fibonacci sequence . . . 2
- 2.1.3 Project tasks . . . 3
- 2.1.4 Provided base project folder . . . 3
- 2.1.5 Getting help from videos . . . 3

2.2 Programming tasks (PT, 30 pts) . . . 3
- 2.2.1 PT 1. Programming the `mp_update_fib_array` function (20 pts) . . . 4
- 2.2.2 PT 2. Programming the `mp_app` function (10 pts) . . . 4
- 2.2.3 C programming concepts used here . . . 5

2.3 Project creation (PC) and management tasks (MT) (20 pts) . . . 6
- 2.3.1 PC 0. Creating project folders and copying files (0 pts) . . . 6
- 2.3.2 PC 1. Generating C code from CubeMX (5 pts) . . . 7
- 2.3.3 MT 1. Managing project in CubeIDE (15 pts) . . . 7

2.4 Debugging tasks (DT, 40 pts) . . . 8
- 2.4.1 Basic debugging techniques to practice . . . 8
- 2.4.2 Step-by-step debugging and programming . . . 8
- 2.4.3 DT 1. Setting up breakpoints (5 pts) . . . 9
- 2.4.4 DT 2. Debugging in the **Debug** perspective (5 pts) . . . 9
- 2.4.5 DT 3. Inspecting local variables (8 pts) . . . 9
- 2.4.6 PT 1. Programming the `mp_update_fib_array` function . . . 10
- 2.4.7 DT 4. Inspecting global variables (8 pts) . . . 10
- 2.4.8 PT 2. Programming the `mp_app` function . . . 10
- 2.4.9 DT 5. Inspecting the memory of global variables (8 pts) . . . 11
- 2.4.10 DT 6. Fixing the bug(s) (6 pts) . . . 11

2.5 Submission of work (10 pts) . . . 12

2.6 Hints for the bug(s) . . . 12

2.1 Introduction

2.1.1 The Fibonacci sequence

The Fibonacci sequence is named after Italian mathematician Leonardo of Pisa, known as Fibonacci. It is a series of unsigned integer numbers where each number is the sum of the two preceding ones. By definition, they start with 0 and 1. Figure 1 shows the first 21 values of the Fibonacci sequence, reproduced from Wikipedia: https://en.wikipedia.org/wiki/Fibonacci_sequence.

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}	F_{18}	F_{19}	F_{20}
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181	6765

Figure 1: Fibonacci sequence from 0 to 20.

2.1.2 An algorithm for generating part of the Fibonacci sequence

The Fibonacci sequence can be easily calculated by using a simple algorithm, as the one shown below, reproduced from Wikipedia:

Algorithm 1: Fibonacci sequence computation and saving

Input : `fib_arr`, pointer to the array to save the Fibonacci sequence
Input : N, the maximum index number of the Fibonacci sequence
Return: Status of execution; Fibonacci sequence is written to `fib_arr` directly

```
ret_val = 0 (success)
if N < 0 then
    ret_val = -1 (error case 1)
else if N == 0 then
    fib_arr[0] = 0
else if N == 1 then
    fib_arr[0] = 0
    fib_arr[1] = 1
else if N <= 20 then
    fib_arr[0] = 0
    fib_arr[1] = 1
    i = 2
    while i <= N do
        fib_arr[i] = fib_arr[i-1] + fib_arr[i-2]
        i = i + 1
    end
else
    ret_val = -2 (error case 2)
end
return
```

2.1.3 Project tasks

In this project, you will perform the following work:

- Write a C function to implement the above algorithm.
- Crete a CubeIDE project to call this function using the provided code, which **may have hidden bugs**.
- Use basic debugging techniques to catch the bug(s) in the provided code using CubeIDE. Note that a hint is provided at the last page of this handout; yet, try not to see the hint during your debugging process.

The above work will be divided into tasks, and each has detailed instructions to finish. You are suggested to read through this handout carefully and discuss with your team members about the basic ideas **before** doing any tasks. Only after you fully understand the ideas provided here can you start with the project.

2.1.4 Provided base project folder

To get started quickly, you can use the provided base project for F412dsc, including all the needed source files, zipped in `ce5d_debug_fib-f412.zip`. You should unzip the file to have the following folder structure:

```
proj_mp
  ce5d_debug_fib
    ce5d_debug_fib_f412dsc
    lib
    renode
    src
```

You need to add the `ce5d_debug_fib_g431n32` folder under `ce5d_debug_fib`, which is used for creating images that run on the G431n32 board for debugging.

2.1.5 Getting help from videos

Try your best to finish the project by reading from this handout and the Companion. Yet, if you get stuck, you can watch a video titled `ce1s-project-creation-management-building-running.mp4`, which walks you through about the creation of a project from scratch, provided for Lab 1.

There are two more videos that are helpful for this project:

- `ce1s-running-code-on-g431n32-with-putty.mp4` for how to run the code using the real MCU hardware together with seeing the results in PuTTY.
- `ce5d-basic-debugging-in-cubeide-f-fib.mp4` for how to perform basic debugging tricks using the real MCU board, including
  - Setting up breakpoints.
  - Viewing local variables using the **Variables** view.
  - Viewing global variables using the **Expressions** view.
  - Viewing the memory directly using a given address.

2.2 Programming tasks (PT, 30 pts)

The main story line of this project is to program a Fibonacci sequence generating function, `mp_update_fib_array`, to produce the first `N+1` values of the Fibonacci sequence for a given in-

teger number `N`. You are required to save these values to an array named `fib_arr` using a C function which can be created based on the above algorithm.

2.2.1 PT 1. Programming the `mp_update_fib_array` function (20 pts)

Below is the prototype of the function, where we pass an array (in the form of a pointer) as the first parameter so that the values of array elements can be changed. We also pass the number of elements (in the form of a value) as the second parameter so that we know how many values to be generated

```
static int mp_update_fib_array(uint32_t *fib_arr, int N)
```

The skeleton of this function is provided in `ce5d_debug_fib.c`, as shown below:

```
static int mp_update_fib_array(uint32_t *fib_arr, int N) {
    int ret_val = 0;   // return value: 0 = success
    if (N < 0) {
        ret_val = -1;
    } else if (N == 0) {
        fib_arr[0] = 0;
    } else if (N == 1) {
        fib_arr[0] = 0;
        fib_arr[1] = 1;
    } else if (N <= 20) {
// Add your code here
    } else {
        ret_val = -2;
    }
    return ret_val;
}
```

You need to program in the section labeled as `//Add your code here` in the above code snippet according to the afore-mentioned algorithm so that in the main function, `mp_app`, you will be able to print the results correctly as compared to the results of Figure 1.

Programming is an iterative work. As such, we need to interleave the programming and debugging tasks that will be discussed below.

2.2.2 PT 2. Programming the `mp_app` function (10 pts)

Above and underneath the snippet of the above `mp_update_fib_array` function in `ce5d_debug_fib.c`, omitted here due to space limit, is the **main code** of the App, as shown below:

```
static uint32_t fibonacci_arr1[10], fibonacci_arr2[13];

// ... the above mp_update_fib_array function goes here

__attribute__((weak))
void mp_app(void) {
    printf("\n\nRunning ce5d_debug_fib App ---\n");

    char result_info[] =
            "Fibonacci number %2d: %4d \t %s \t 0X%02X\n";
    char bin_str[50];
    int dec_int = 0;
```

```
    int N[] = {-1, 0, 1, 5, 21};

    for (int i = 0; i < sizeof(N)/sizeof(N[0]); i++) {
        int call_status = mp_update_fib_array(fibonacci_arr1, N[i]);
        printf("Status of calling update_fib_array with N=%d is %d\n",
                N[i], call_status);
    }

    mp_update_fib_array(fibonacci_arr1, 5);
    printf("\n--- 5 Fibonacci numbers from array 1: ---\n");
    for (int i = 1; i <= 5; i++) {
        dec_int = fibonacci_arr1[i];
        mp_bin_num_2_n_bit_vrbs_bin_str(bin_str, dec_int, 8);
        printf(result_info, i, dec_int, bin_str, dec_int);
    }

    mp_update_fib_array(fibonacci_arr1, 10);
    mp_update_fib_array(fibonacci_arr2, 13);

    printf("\n--- Fibonacci numbers from array 1: ---\n");
    for (int i = 1; i <= 10; i++) {
        dec_int = fibonacci_arr1[i];
// Add your code here
    }

    printf("\n--- Fibonacci numbers from array 2: ---\n");
    for (int i = 1; i <= 13; i++) {
        dec_int = fibonacci_arr2[i];
// Add your code here
    }

    while (1) {
    }
}
```

You need to **add your code to the two locations** which are commented as `//Add your code here` to print the results from the function correctly as compared to the results of Figure 1. Expected results will be given below when we discuss topics of step-by-step debugging and programming later.

Often times, we introduce bugs in programming. While some bugs are easy to find out, others can be difficult. Intentionally, we created a hidden bug in the provided *main code*. You may be able to spot the bug immediately; just pretend that you did not and follow along with the instructions until the last step where you need to fix the bug.

2.2.3 C programming concepts used here

In this project, we review and practice the following basics in C programming:

- Writing C functions with two different approaches for passing parameters, as given in the above prototype of the function:
  - Passing parameters by references, like `fib_arr` above.
  - Passing parameters by values, like `N` above.
- Using program variables in two different scopes, as shown in the provided *main code* shown below:

  - Using (private) global variables.
  - Using local variables.

- Using the `while` and `for` loops to control the flow of the program. A loop has to be used in the function to generate `N` values for a given `N`.
- Using the `if` statement to handle different situations. The value of `N` has to be checked to perform the needed calculations.
- Using pointer or array index to access array elements. Now, we can focus on the latter. For example, for the above `fib_arr`, we can use `fib_arr[2]` to access element 2 in `fib_arr`. **Recall that C arrays start with index 0**.

If you need some help with basic C programming, please refer to the C MiniRef chapter of the Companion or talk to the TAs.

2.3 Project creation (PC) and management tasks (MT) (20 pts)

We have practiced the creation of the project from `.ioc` file using CubeMX previously. Here we will do it again. You are suggested to work it out following the instructions without watching the video. Think about why it works in the instructed way. When you get stuck, watch the provided video.

2.3.1 PC 0. Creating project folders and copying files (0 pts)

You can skip most of this step as the project folder is provided together with four of the five subfolders underneath; only `ce5d_debug_fib_g431n32` has to be created. For reference, here are the steps you can follow (with the ones you must do in **bold**):

- Create a folder named `ce5d_debug_fib` directly under `proj_mp` and create the following two folders each for a single MPB:
  - `ce5d_debug_fib_f412dsc`.
  - `ce5d_debug_fib_g431n32`. (**This one must be created**.)
- Copy the `.ioc` file for G431n32 from the corresponding subfolders of a demo project, such as `ce1s_print_in_diff_formats`. (**The one for the G431n32 board must be copied**.)
- Copy the `lib` and `renode` folders from `ce1s_print_in_diff_formats`.
  - There is no need to change anything in the `lib` folder, although we can remove the two files named `mp_str_dec_number` as they are not used in this project.
  - In `renode`:
    * Rename `debug_ce1s_print_in_diff_formats_f412dsc.resc` to `debug_ce5d_debug_fib_f412dsc.resc`.
    * Change each `ce1s_print_in_diff_formats` in the `debug_ce5d_debug_fib_f412dsc.resc` file to `ce5d_debug_fib`.
- Create a folder named `src` and copy the enclosed `ce5d_debug_fib_app.c` file of this handout to it. Also copy the `_mp_main.c` from the `src` folder of `ce1s_print_in_diff_formats` to it.

2.3.2 PC 1. Generating C code from CubeMX (5 pts)

We assume `ce5d_debug_fib_g431n32` is created and a `.ioc` file for G431n32 has been placed underneath it. Now, do the following:

- Double-click the `.ioc` file in `ce5d_debug_fib_g431n32`, and CubeMX will run.
- Save the project to a new one by clicking **[File » Save Project as]**. As the project folder is created correctly, we just need to click **[OK]** to move forward.
- Click **[Generate Code]**, we can have the code generated.
- At the end, click **[Open project]**, and CubeIDE will run.
- Click **[Ok]** when you see a message saying that the project is successfully imported.

2.3.3 MT 1. Managing project in CubeIDE (15 pts)

Detailed steps have been practiced in Lab 1. Here we provide them again so that the handout will be self-contained. Note that we illustrate the steps using Windows, but they work on macOS similarly.

Step 1: Add a hook function for user code. Open up the `main.c` file under `Application/User` of the `ce5d_debug_fib_g431n32` project in CubeIDE. Copy the following two lines of code

```
void mp_main(void);   // <-- Prototype of the HOOK
mp_main();            // <-- The HOOK function
```

to between the following comments:

```
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */
```

Step 2. Add a source folder to the project: `lib_src`. This is done by right-clicking on the project title in CubeIDE and choose **[New » Folder]**.

Step 3. Include header files to the project located in the `lib` folder (and the `src` folder if applicable). This is done by right-clicking the project title and choose **[Properties]**. A **[Properties for . . .]** dialog will pop up.

- In the left pane of the dialog, click **[C/C++ General » Paths and Symbols]**. The right pane will be the **Paths and Symbols** pane.
- In this pane, click the **Includes** tab.
- Under **Languages**, choose **[GNU C]**.
- In the right pane, we should be able to see all the *include directories*. Click **[Add ..]** to open a **Add directory path** dialog.
- Type `../../lib` in the edit field of this dialog.
- Check the checkboxes of **Add to all configurations**.
- Click **[Ok]**.

Step 4. Add linked files to the project. Here, we need to

- Open up a File Explorer in Windows (or Finder on macOS). Drag `mp_bin_str_printing.c` and `mp_uart_redirect.c` in the `lib` folder to the recently created `lib_src` folder. Choose Link, not Copy, in the pop up dialog.

- Repeat the same to drag `ce5d_debug_fib_app.c` in the `src` folder to the recently created `lib_src` folder. Choose Link, not Copy.

Now, we can click the **[Build]** button (the one with a hammer) to build the project. You should be able to build the project without problem. If you have, contact the TAs to fix the issue. If everything is going well, you can move to the next step.

2.4 Debugging tasks (DT, 40 pts)

2.4.1 Basic debugging techniques to practice

Programming and debugging always proceed in an **iterative way**. You need to write a block of code, then test it using some debugging techniques. When there is no compiler error and the results are the same as what you expect, move on to write some more code and debug it, until the entire project is finished.

Note that sometimes you will see warnings. They should be handled accordingly.

- If you see something like `warning: unused variable 'bin_str'`, you can ignore it as you have not added all your user code yet.
- If you see other warnings, please record what you see and report to the instructor.

To debug the code using debugger in CubeIDE, we **need to use the NUCLEO-G431KB Board**. To see the print out results with G431n32, we need to use PuTTY. Before moving on, read the VCP chapter of the Companion for basic operations of this tool; or watch the corresponding video about how to use PuTTY.

There are a number of basic debugging techniques we can use in this lab, all in the **Debug** perspective:

- Halting the program using breakpoints.
- Displaying the local variables in the **Variables** view.
- Displaying the global variables in the **Expression** view.
- Displaying the global variables in the **Memory** view.

Watch the video about how to use CubeIDE for debugging. Or, you can refer to the Debugging with CubeIDE section of the CubeIDE chapter of the Companion for the basic usages of debugging.

Note that you need to go back and fourth between the **Edit** and **Debug** perspectives of CubeIDE frequently during debugging.

Again, we will use the G431n32 board for the debugging. Click `ce5d_debug_fib_g431n32` in CubeIDE to make sure this is the active project for building and debugging.

Before moving on, you need to ask the TA for an MCU board if you have not done so yet. Plug in the board to your computer using a Micro-USB cable. You may be asked to upgrade the firmware. If so, just do it. Now, assume the hardware is connected.

2.4.2 Step-by-step debugging and programming

Below, we will put the debugging and programming tasks together in a step-by-step manner to finish this project.

2.4.3 DT 1. Setting up breakpoints (5 pts)

Let's start with debugging tasks (DT) by setting up breakpoints.

First, make sure we are in the **editor perspective**. When we start up CubeIDE, we are in the **Editor** perspective by default. If we wan to go to the **Editor** perspective from another perspective, we can click the **[C/C++]** icon at the upper right corner of CubeIDE.

Then, we need to open up the main user code `ce5d_debug_fib_app.c` in the **editor perspective**. This can be done by double-clicking this file under `lib_src`.

Now, we set up a breakpoint at each of the following line of code of `ce5d_debug_fib_app.c`:

- BP1: `printf("\n\nRunning ce5d_Debug_Fib---\n");`
- BP2: `mp_update_fib_array(fibonacci_arr1, 5);`
- BP3: `mp_update_fib_array(fibonacci_arr1, 10);`
- BP4: `while (1){` (or the blank line before this line)

To set up a breakpoint, we just need to double click at the vertical bar, called the **Gutter**, which is immediately to the left of the line number we want to break. You will see a breakpoint icon at each breakpoint.

Note that if you have been using CubeIDE for debugging other projects, there may be leftover breakpoints from other projects. They should be removed. Ask the instructor for an illustration in class.

2.4.4 DT 2. Debugging in the Debug perspective (5 pts)

To go to the **Debug** perspective from the editor perspective, click the **[Debug]** icon (the one looks like a bug). See the video for a quick walkthrough. If you want to go beyond the video, please refer to the Debugging with CubeIDE section of the CubeIDE chapter of the Companion.

Now, run the code by clicking the **[Run]** button and let it halt at BP1. You should be able to see the line of the code at the breakpoint is highlighted. Take a screenshot of the halted program; this is **Artifact 1**.

2.4.5 DT 3. Inspecting local variables (8 pts)

Now, ensure the program is halt at BP1. We will inspect the element values of array `N[]`, which is a local array defined in `mp_app`, using the **Variables** view in the **Debug** perspective.

The **Variables** view is used to show the local variables, which can only be seen when the program is halt inside the function where the variables are defined. To use this view, go to the right pane of the **Debug** perspective and click the **Variables** tab. Click the Right arrow to the left of `N` to see the values of its elements.

Since the code for initializing array `N[]` has not run yet, you should find that the values of `N[]` are different than what to be initialized in the code.

Now, resume the running of the program by clicking the **[Run]** button again. The program should stop at BP2.

You should be able to find that the values of `N[]` are the same as what are initialized. Take a screenshot of these values; this is **Artifact 2**.

Take a look at the printed results in PuTTY. It should be the same as what is shown below:

```
Running ce5d_debug_fib App ---
Status of calling mp_update_fib_array with N=-1 is -1
Status of calling mp_update_fib_array with N=0 is 0
Status of calling mp_update_fib_array with N=1 is 0
Status of calling mp_update_fib_array with N=5 is 0
Status of calling mp_update_fib_array with N=21 is -2
```

Then, we can shift gear to do some programming tasks (PT) with some requirements as appropriate.

2.4.6 PT 1. Programming the `mp_update_fib_array` function

This is the actual work of the PT 1. Programming the `mp_update_fib_array` function (20 pts) section.

The input parameters and output of the `static int mp_update_fib_array(uint32_t *fib_arr, int N)` function are explained below:

- Input parameters:
  - `fib_arr` is a pointer, which is the address of `fib_arr`, the array used to save the Fibonacci sequence. **Note that the name of an array is a pointer to the array or element 0 of the array**.
  - `N`, the number of elements in the Fibonacci sequence we need to generate, including the first `0`.
- Output: `status`, an indicator to the running of the program. We do not return the values to the array; we change the values of `fib_arr` directly in the function.

When the program is well written, if we halt the program at BP3, we should see the following:

```
--- 5 Fibonacci numbers from array 1: ---
Fibonacci number  1:   1       0b0000_0001     0X01
Fibonacci number  2:   1       0b0000_0001     0X01
Fibonacci number  3:   2       0b0000_0010     0X02
Fibonacci number  4:   3       0b0000_0011     0X03
Fibonacci number  5:   5       0b0000_0101     0X05
```

Note that we have displayed the values in decimal, binary, and hexadecimal.

2.4.7 DT 4. Inspecting global variables (8 pts)

DT 4. (10 points) Now, assume we are at BP3. Take a screenshot of the values of `fibonacci_arr1` using the **Expression** view in the **Debug** perspective, which is at the right of the **Variables** view. This is **Artifact 3**.

The **Expression** view is used to display global variables, and it is suitable to see `fibonacci_arr1` which is defined as a global. Note that to view a global variable, we need to click the **[Add new expression]** field and type the global variable's name.

2.4.8 PT 2. Programming the `mp_app` function

This is the actual work of the PT 2. Programming the `mp_app` function (10 pts) section.

Add code in the two `for` loops to print out the results like below:

```
--- Fibonacci numbers from array 1: ---
Fibonacci number  1:    1   0b0000_0001   0X01
Fibonacci number  2:    1   0b0000_0001   0X01
Fibonacci number  3:    2   0b0000_0010   0X02
```

```
Fibonacci number  4:    3   0b0000_0011   0X03
Fibonacci number  5:    5   0b0000_0101   0X05
Fibonacci number  6:    8   0b0000_1000   0X08
Fibonacci number  7:   13   0b0000_1101   0X0D
Fibonacci number  8:   21   0b0001_0101   0X15
Fibonacci number  9:   34   0b0010_0010   0X22
Fibonacci number 10:    0   0b0011_0111   0X00
--- Fibonacci numbers from array 2: ---
Fibonacci number  1:    1   0b0000_0001   0X01
Fibonacci number  2:    1   0b0000_0001   0X01
Fibonacci number  3:    2   0b0000_0010   0X02
Fibonacci number  4:    3   0b0000_0011   0X03
Fibonacci number  5:    5   0b0000_0101   0X05
Fibonacci number  6:    8   0b0000_1000   0X08
Fibonacci number  7:   13   0b0000_1101   0X0D
Fibonacci number  8:   21   0b0001_0101   0X15
Fibonacci number  9:   34   0b0010_0010   0X22
Fibonacci number 10:   55   0b0011_0111   0X37
Fibonacci number 11:   89   0b0101_1001   0X59
Fibonacci number 12:  144   0b1001_0000   0X90
Fibonacci number 13:  233   0b1110_1001   0XE9
```

To see the above, you need to run the code so that it should halt at BP4. Take a screenshot of the above; this is **Artifact 4**.

2.4.9 DT 5. Inspecting the memory of global variables (8 pts)

DT 5. (10 points) Now, you should be able to see the addresses of `fibonacci_arr1` and `fibonacci_arr2` using the **Expressions** view; the address is under the **Value** of the array, with something like `0x2000mnop`, where each char/letter of `mnop` is a hexadecimal digit.

Now, use the lower address of the above two array to open the **Memory** view to inspect the array values there. Use the *Unsigned Integer* format for rendering. Take a screenshot; this is **Artifact 5**.

The **Memory** view is at the lower pane of the **Debug** perspective. You need press the **[+]** icon to open a memory monitor. You need to input the lower memory address. Note that you need to use Hexadecimal when inputting the address as this is what we obtain from the **Expressions** view.

Later on, we will see more display options using the memory view.

2.4.10 DT 6. Fixing the bug(s) (6 pts)

Now, you may have figured out what is wrong with the print out of

```
10th Fibonacci number:   0       0b0000_0000     0X00
```

for `fibonacci_arr1`.

If yes, please change the code and print out the correct results. Take a screenshot as **Artifact 6**. If not, you can use some of the debug tricks to find the bug. If you get stuck, just relax and come back to the project again at a later time. If you still do not get it after a few tries, take a look at the last page of this handout for hints.

2.5 Submission of work (10 pts)

The focus of this lab is programming and debugging. Yet, we need to see the report to assess your work. The allocated 10 points for the report is for the writing and format only. You will lose the points of the lab tasks if your results are not correct or the screenshots are missing.

The following are the requirements for the report:

- Include **Artifact 1** and describe how do you know that the program has stopped at BP1.
- Include **Artifact 2** and determine the size of array `N` from it.
- Include **Artifact 3** and determine the size of array `fibonacci_arr1` from it.
- Include **Artifact 4**.
- Include **Artifact 5** and estimate the size of `fibonacci_arr2` from it.
- Include **Artifact 6**.

Please team up with another student to work on the lab together. Yet, **you need to submit your own lab report for this lab**. This is a good way to accumulate basic debugging skills.

The report and zip file names should be something like:

- `ce5d-report-lastname-firstname.pdf`.
- `ce5d-proj-lastname-firstname.zip`.

Before zipping your project, clean out the builds you have created in CubeIDE.

2.6 Hints for the bug(s)

This is an issue that happens very often in real applications. Intentionally, we don't want to provide details since we wanted you to *struggle* a bit so that you will not make this type of mistakes often.

Here is a little hint: The index of the array in C starts at 0, and hence the size of the array is the maximum index number plus 1.

If you need more hints, please talk to the TAs or the instructor.